QUANTUM LEAP

BUILDING REAL QUANTUM PROGRAMS WITH PYTHON



Beilak Aliev
Team Lead

BEILAK ALIEV

- 12+ years in software engineering
- Team Lead on weekdays
- Quantum computing researcher on Saturdays





HISTORY



QUANTUM MILESTONES

1980	Yuri Manin
1982	Richard Feynman
1994	Peter Shor: factoring algorithm
1996	Grover's search: $O(\sqrt{N})$
2022	433 qubits
2023	1121 qubits

WHERE CAN QUANTUM COMPUTING BENEFIT BUSINESS?



BENEFIT

Industry	Business problems	Potential value
Pharmaceuticals	Drug discovery, molecular simulation, chemical reactions	Faster research and development of new medicines
Materials	Battery development, new materials, chemical catalysts	Discover better materials
Finance	Risk analysis, Monte Carlo simulations, portfolio optimization	Faster risk calculations and more complex financial models
Logistics	Route planning, scheduling, resource allocation	Better use of resources
AI	Classification, anomaly detection	Potential advantages in specific ML tasks

QUANTUM ADVANTAGE



QUANTUM ADVANTAGE

Quantum computers solve specific types of problems.

Problem type	Quantum algorithms	Potential advantage
Quantum Simulation	VQE, Hamiltonian Simulation	Model molecules and materials that are difficult to simulate classically
Search	Grover's Algorithm	Reduce search complexity from (O(N)) to (O(√N))
Monte Carlo & Probability	Amplitude Estimation	Reduce complexity
Factorization	Shor's Algorithm	Exponential speedup over the best known classical factoring methods, in terms of input length
Optimization & ML	QAOA, QNN, VQC	Potential gains for specific problems; no general advantage demonstrated yet

MINI-THEORY



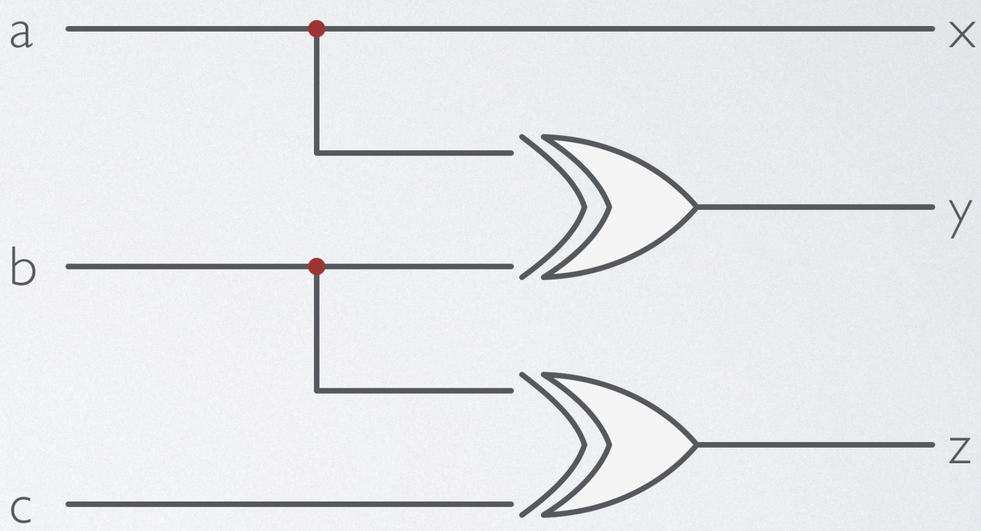
CLASSICAL BITS

- A bit stores 0 or 1
- Voltage: low or high
- Switch: off or on



CLASSICAL LOGIC

- Logic gates transform bits
- XOR outputs 1 when inputs differ

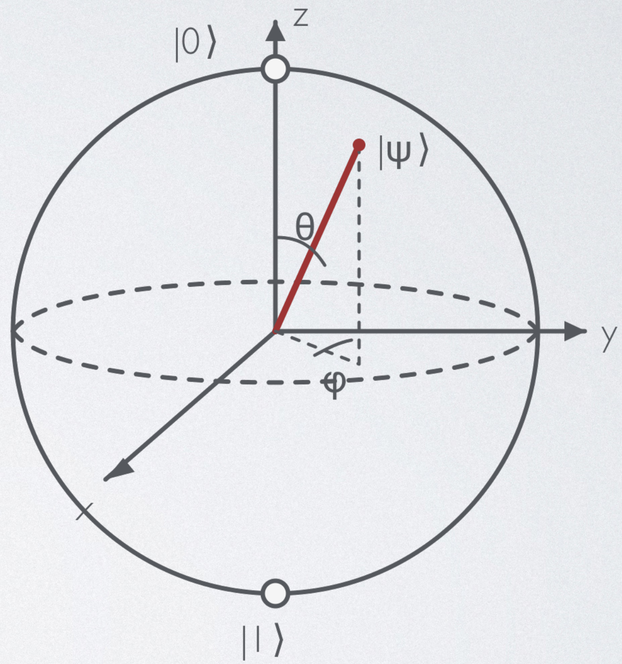


QUBITS

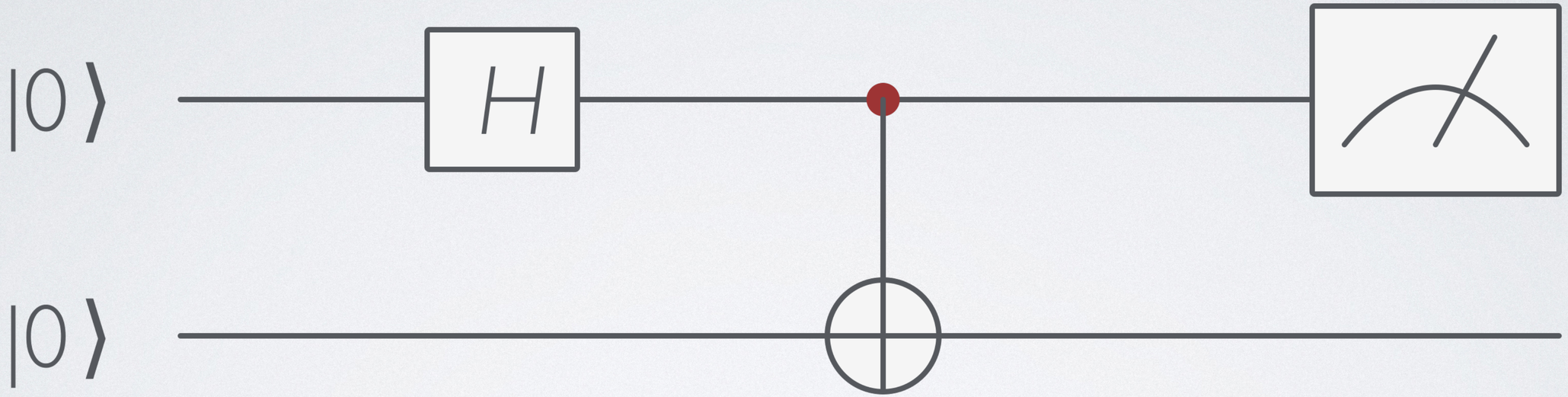
- 0 and 1 are basis states
- Superposition combines both
- Measurement yields 0 or 1

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

$$|1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$



QUANTUM CIRCUIT



GATES AS MATRICES

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

$$CX = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$$

$$CCX = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{pmatrix}$$

QUANTUM PROGRAMMING ECOSYSTEM



QUANTUM PROGRAMMING ECOSYSTEM

Languages

- Python
- C++
- Q#
- OpenQASM
- Quil

Libraries

- Qiskit Aer (Simulation)
- OpenFermion (Quantum chemistry)
- QuTiP (Quantum physics)

Frameworks & SDKs

- Qiskit (IBM)
- Cirq (Google)
- PennyLane (Xanadu)
- QDK (Microsoft)
- Amazon Braket (AWS)
- Ocean SDK (D-Wave)

PRACTICE



QUANTUM HELLO WORLD

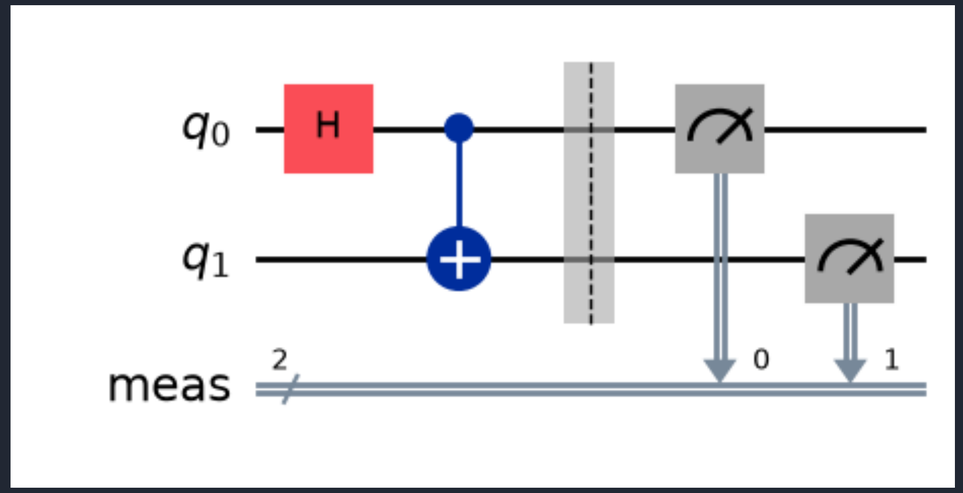
QUANTUM HELLO WORLD

```python
from qiskit import QuantumCircuit

# Circuit
qc: QuantumCircuit = QuantumCircuit(2)

qc.h(0) # Hadamard gate
qc.cx(0, 1) # Controlled not gate

qc.measure_all()
qc.draw(output="mpl")
```

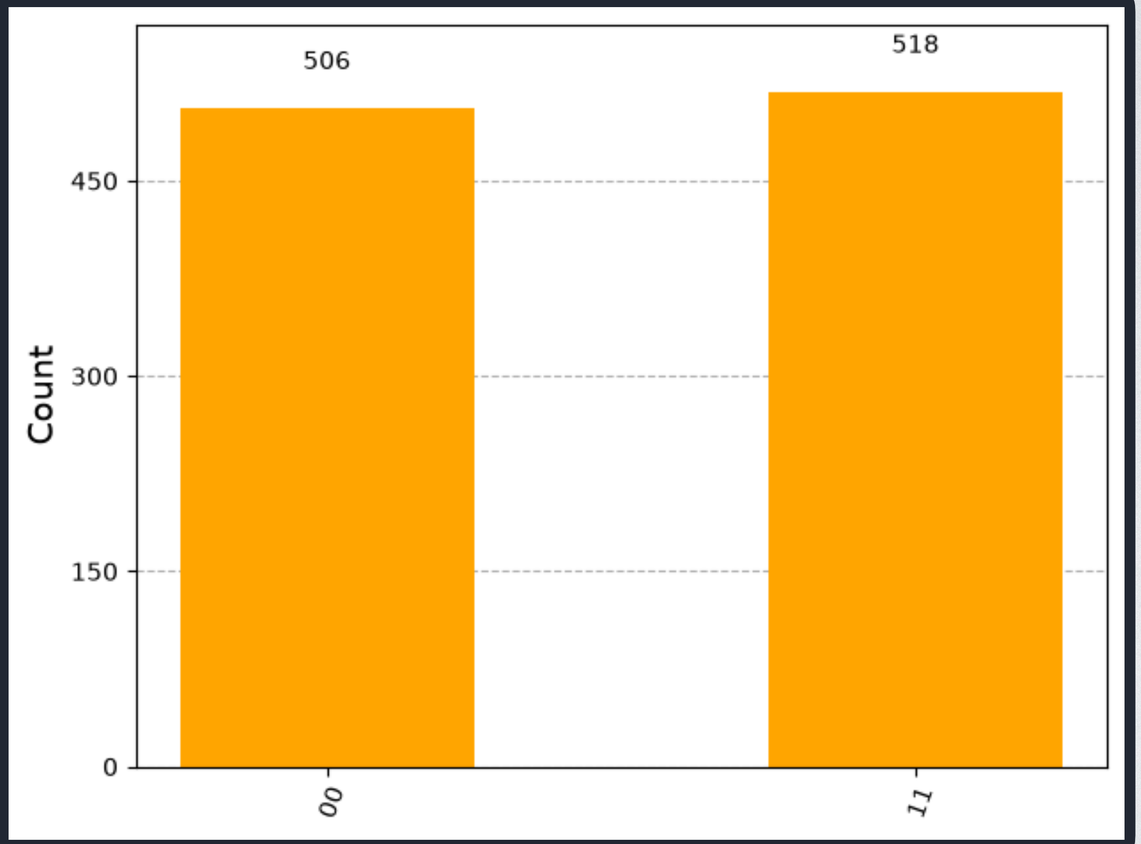


MEASURE Q HELLO WORLD

```python
# SIMULATOR
from qiskit_aer.primitives import SamplerV2
from qiskit.visualization import plot_histogram

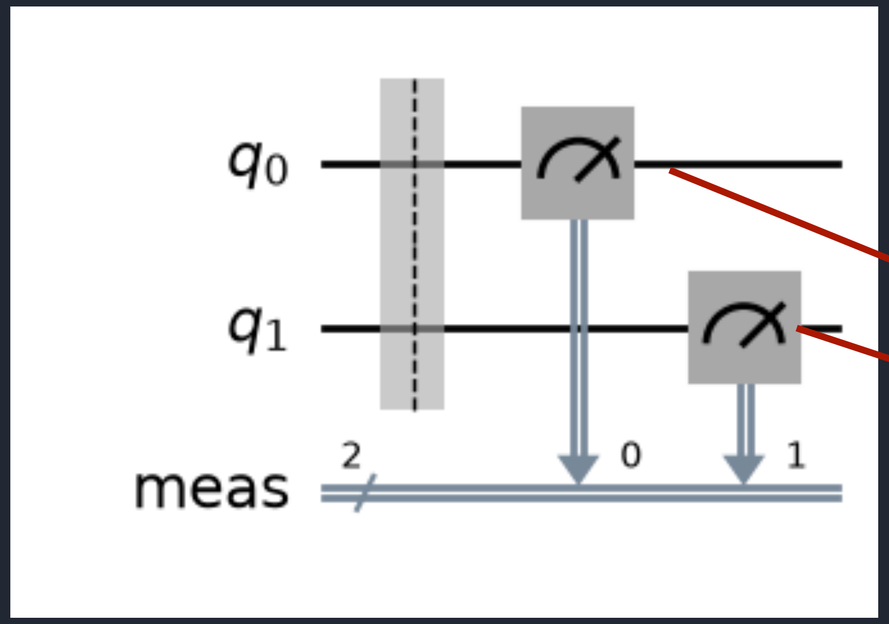
sampler = SamplerV2()
job = sampler.run([qc], shots=1024)
sim_result = job.result()
sim_counts = sim_result[0].data.meas.get_counts()

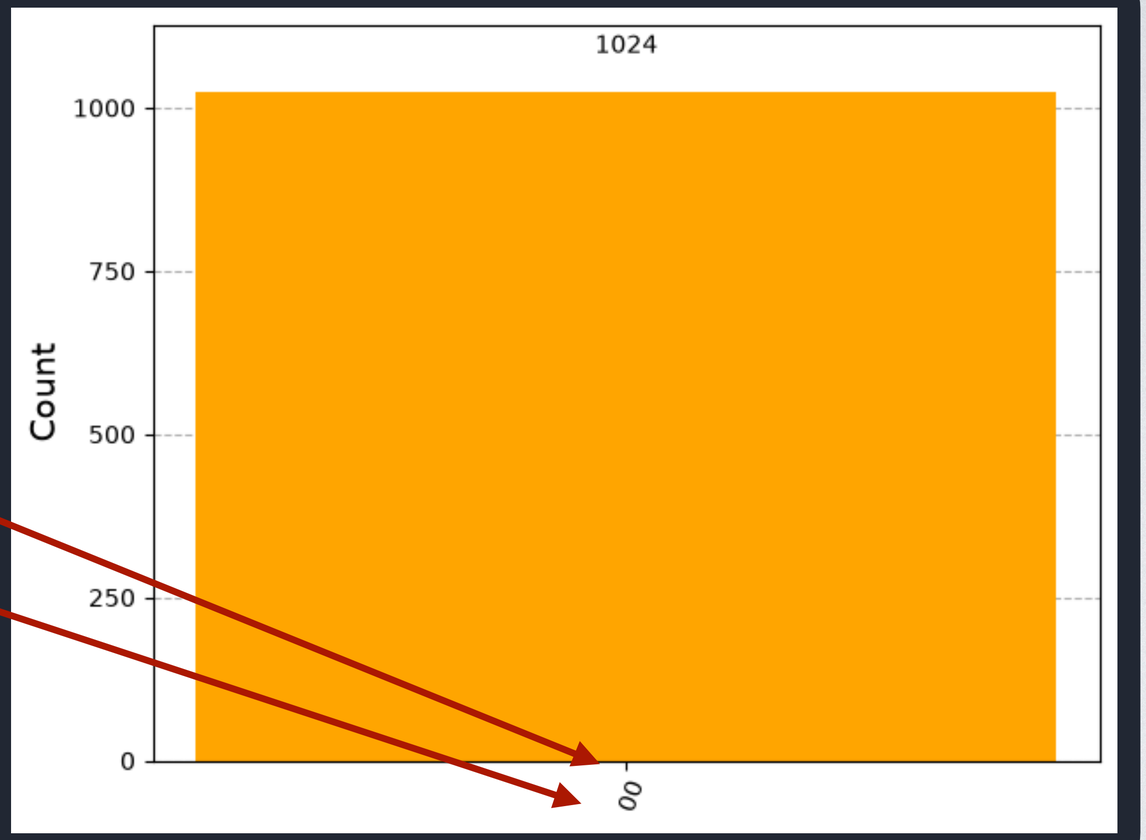

plot_histogram(sim_counts, color="orange")
```



QUANTUM HELLO WORLD

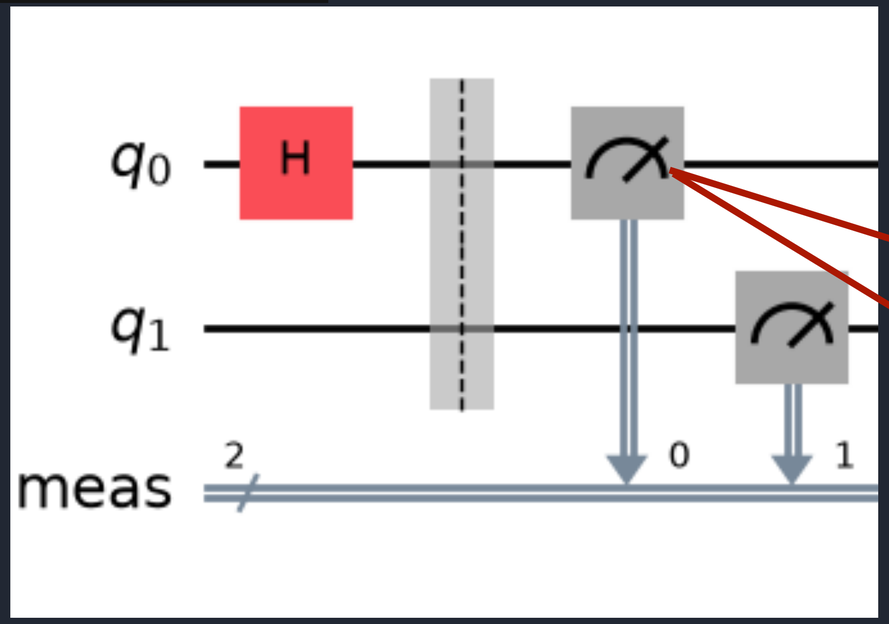
```
# Circuit
qc: QuantumCircuit = QuantumCircuit(2)
qc.measure_all()
```

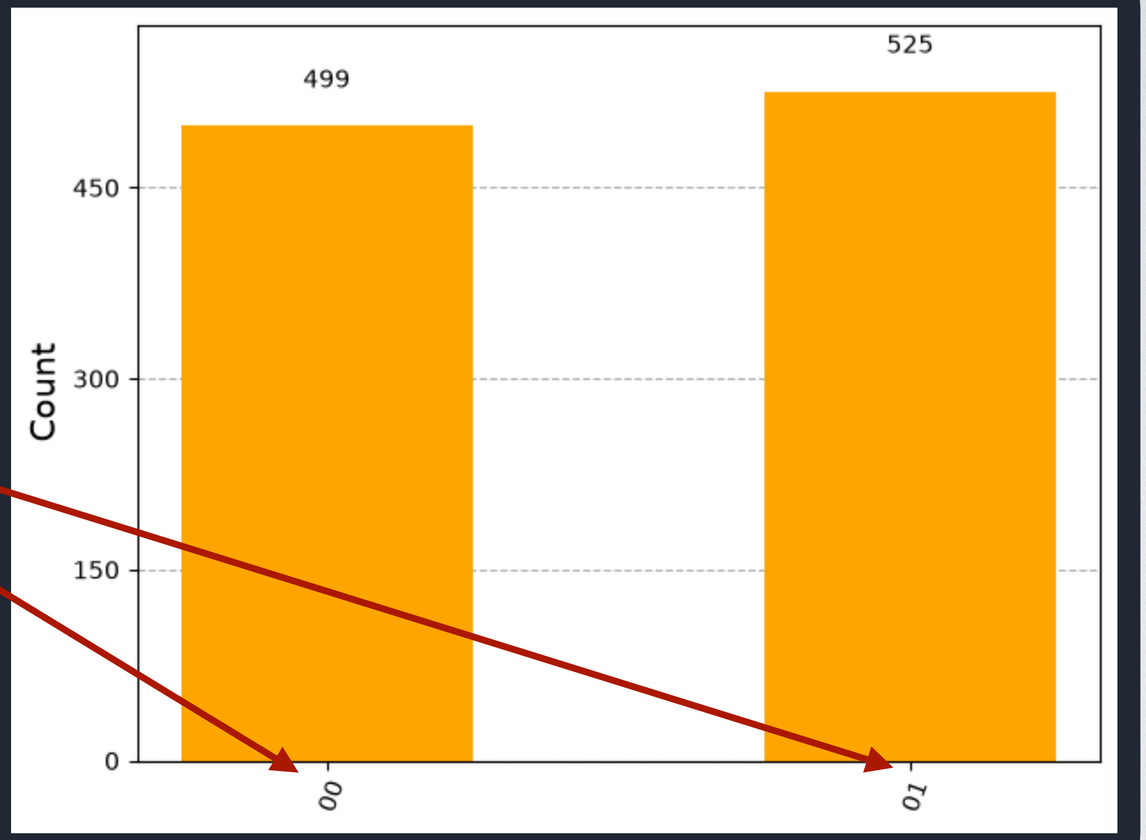




QUANTUM HELLO WORLD

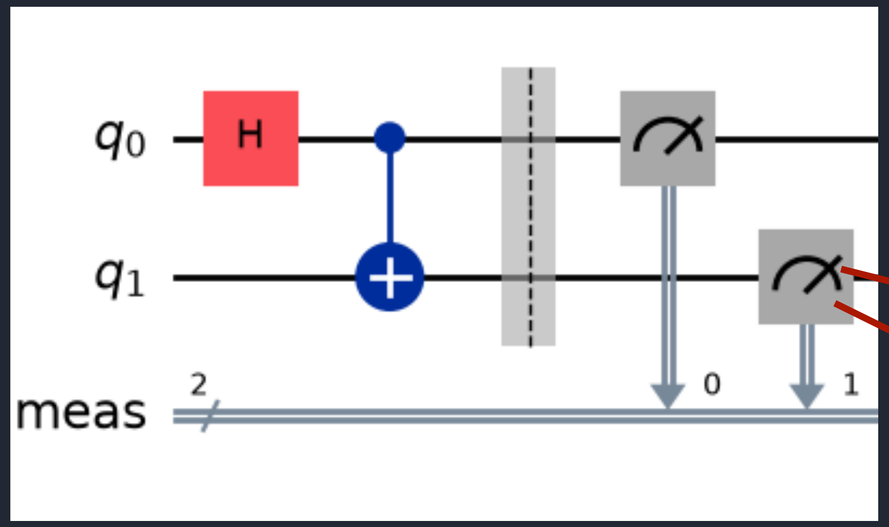
```python
# Circuit
qc: QuantumCircuit = QuantumCircuit(2)
qc.h(0) # Hadamard gate
qc.measure_all()
```

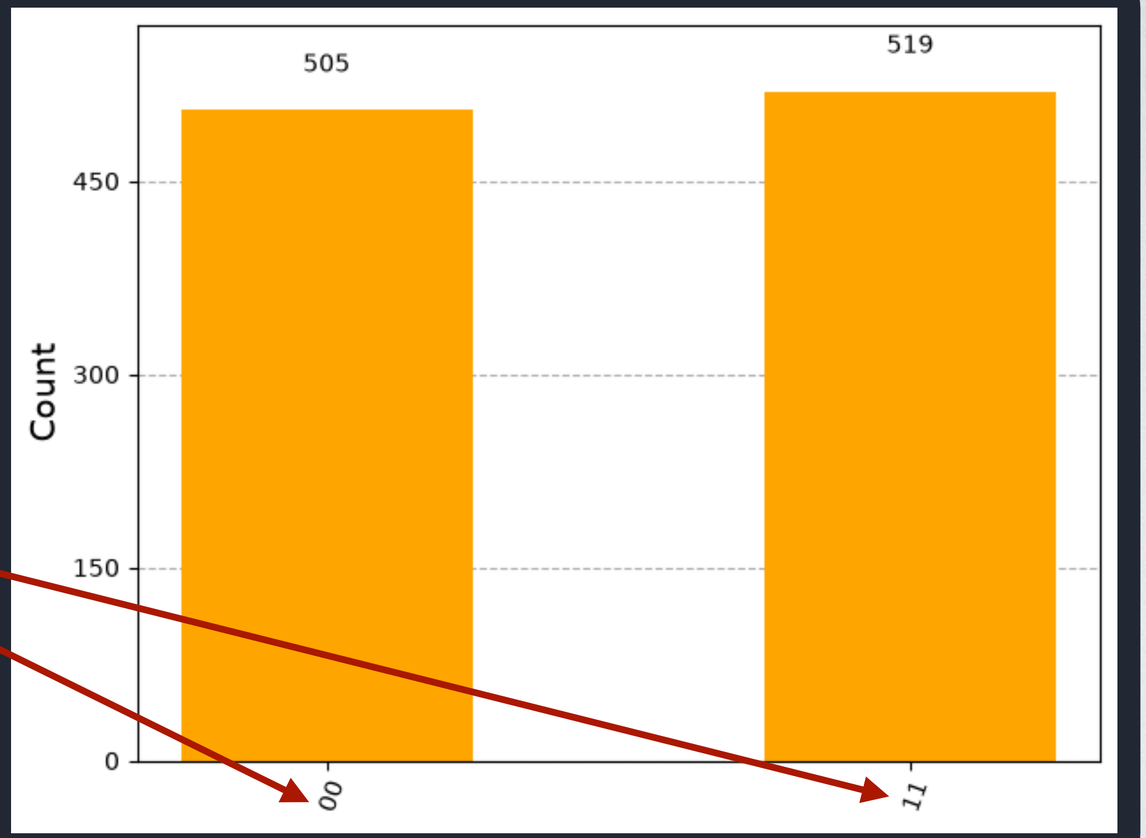




QUANTUM HELLO WORLD

```python
# Circuit
qc: QuantumCircuit = QuantumCircuit(2)
qc.h(0) # Hadamard gate
qc.cx(0, 1) # Controlled not gate
qc.measure_all()
```





RUN ON REAL HARDWARE

Quantum hello world

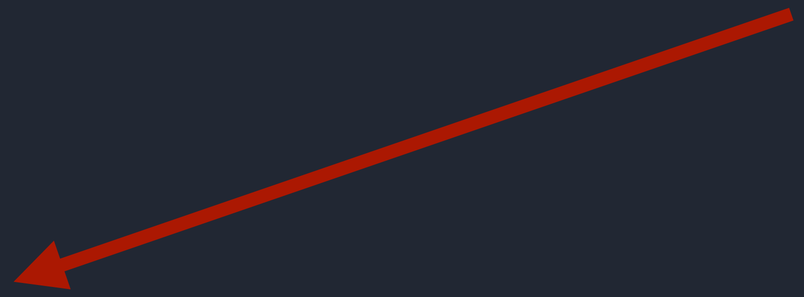
FIND QPU

```python
from qiskit_ibm_runtime import QiskitRuntimeService

service = QiskitRuntimeService(
    channel="ibm_quantum_platform",
    token=TOKEN,
    instance='auto',
)

# 1. Select a real quantum processor
backend = service.least_busy(
    operational=True,
    simulator=False,
    min_num_qubits=2,
)
print(f"Selected QPU: {backend.name}")
```

```
Selected QPU:
ibm_marrakesh
```



```python
from qiskit.transpiler import generate_preset_pass_manager

# 2. Adapt the circuit to IBM hardware
pm = generate_preset_pass_manager(
    backend=backend,
    optimization_level=1,
)
isa_qc = pm.run(qc)

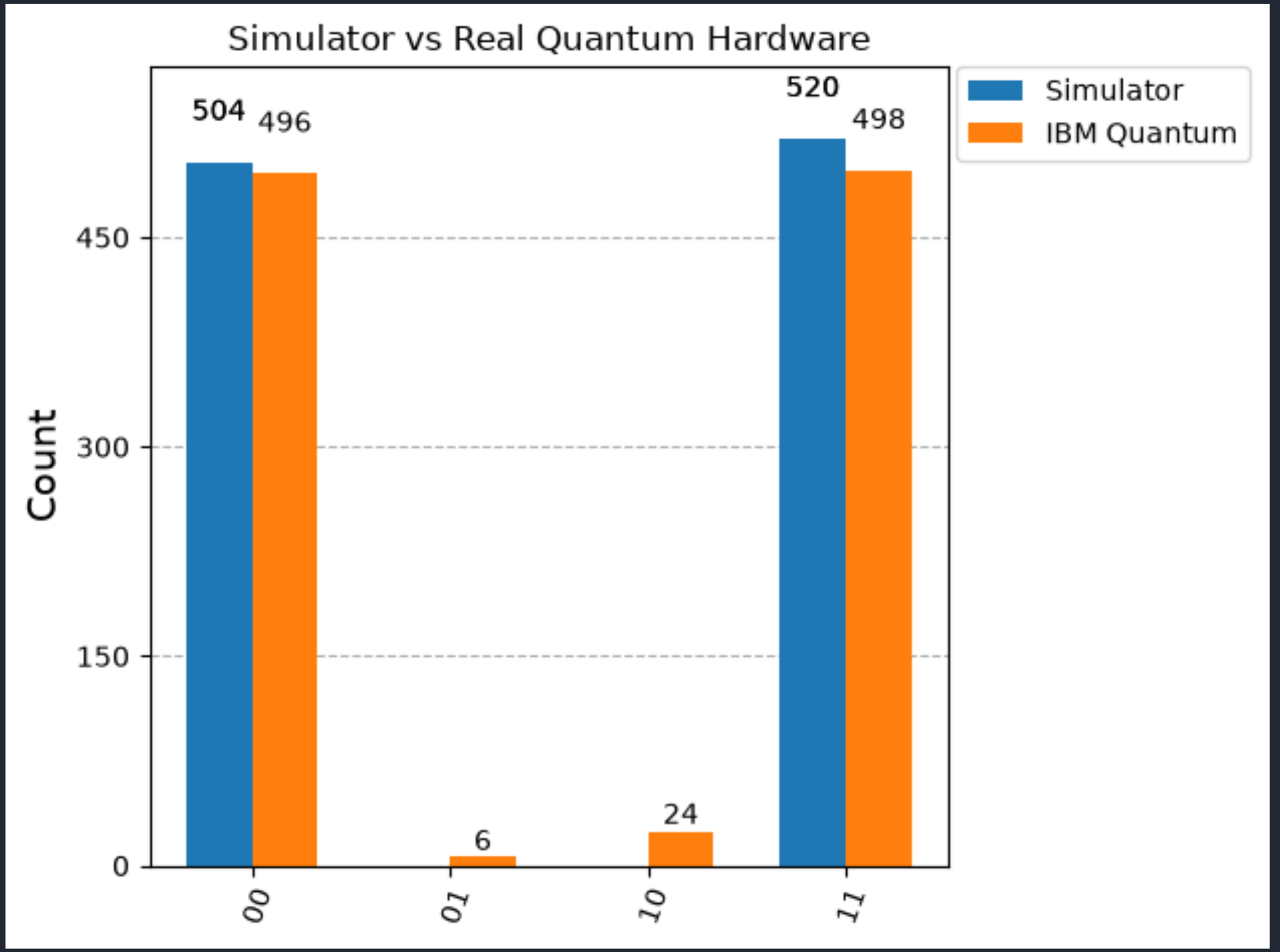

# 3. Execute on a real quantum computer
sampler = IBMSampler(mode=backend)
job = sampler.run([isa_qc], shots=1024)

print(f"Job ID: {job.job_id()}")
```

Job ID:
db75vbur6oqs7386r6tg

My recent workloads

ID	Status	Instance	Created ↓	QPU
db7872iraffc73f...	Pending	open-instance	10/13/26, 10:16 PM	ibm_fez
db76242raffc73...	Completed	open-instance	10/13/26, 7:49 PM	ibm_kingston
db75vbur6oqs7...	Completed	open-instance	10/13/26, 7:43 PM	ibm_marrakesh


Simulator vs Real Quantum Hardware
Simulator
IBM Quantum
Count
0
150
300
450
504
496
6
24
520
498
00
01
10
11

VQE

VARIATIONAL QUANTUM EIGENSOLVER

A hybrid quantum-classical algorithm for finding minimum-energy states.

- Quantum part: Prepares quantum states and measures their energy.
- Classical part: Optimizes the quantum circuit parameters to minimize energy.

Applications: Optimization problems

VQE

Step-by-Step

VQE STEP 1

$$\max f(a,b) = 3a + 2b - 4ab$$

Define the Problem

$$a, b \in 0,1$$

VQE STEP 2

Build a Quadratic Program

Describe the variables and the objective function in a structured mathematical model that would be converted into Qiskit.

```
from qiskit_optimization import QuadraticProgram

small_problem = QuadraticProgram("Two-qubit VQE")
small_problem.binary_var("a")
small_problem.binary_var("b")
small_problem.maximize(
   linear={"a": 3, "b": 2},
   quadratic={("a", "b"): -4}
)

print(small_problem.prettyprint())
```

```
Problem name: Two-qubit VQE

Maximize
  -4*a*b + 3*a + 2*b

Subject to
  No constraints

  Binary variables (2)
    a b
```

VQE STEP 3

Convert to Ising

Transform the optimization problem into an energy-minimization problem represented by a Hamiltonian.

```
cost_operator, _ = small_problem.to_ising()
```

VQE STEP 4

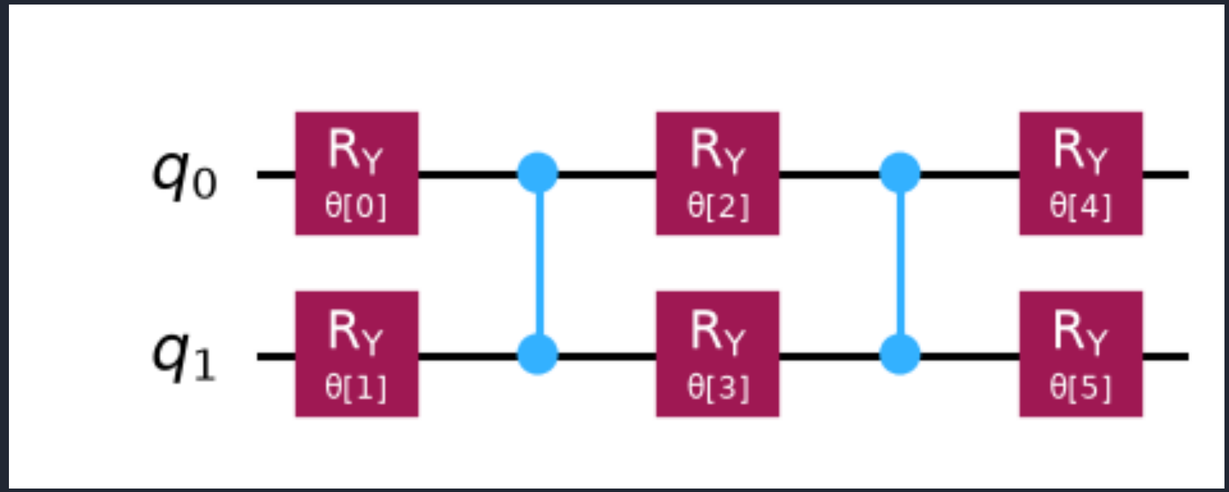
Create a Quantum Circuit

Build a parameterized circuit (ansatz) that prepares candidate quantum states.

```
from qiskit.circuit.library import n_local

small_ansatz = n_local(
  2, "ry", "cz", reps=2,
  entanglement="full",
)

small_ansatz.draw(output="mpl", scale=1.1)
```



VQE STEP 5

Run Hybrid Optimization

Quantum measurements estimate energy. COBYLA updates circuit parameters to minimize it.

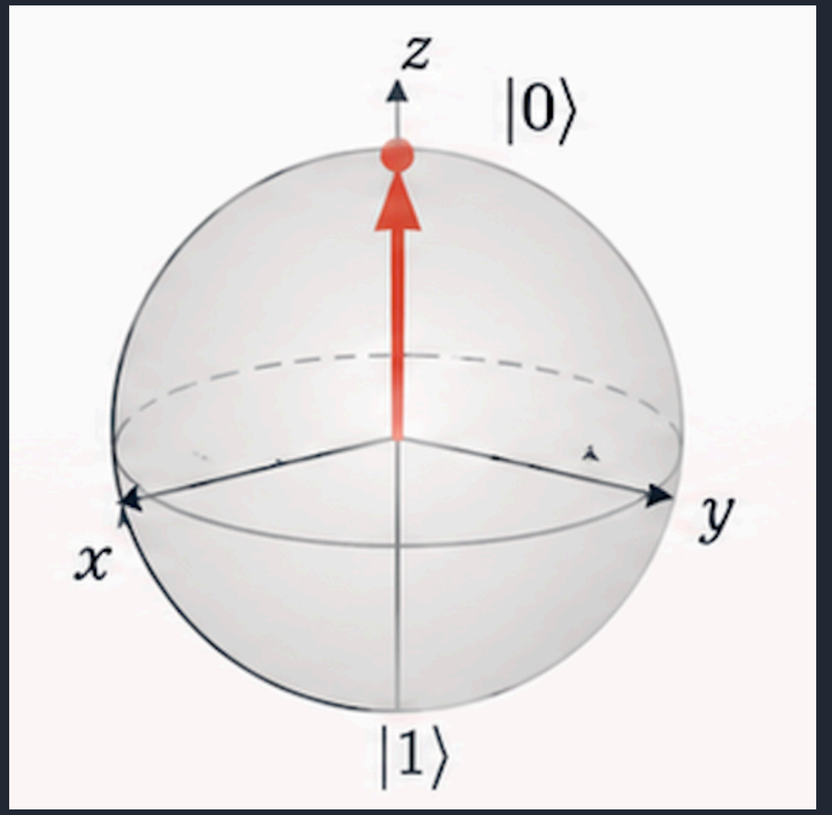
```
from qiskit.primitives import
StatevectorSampler
from qiskit_algorithms import SamplingVQE


small_vqe = SamplingVQE(
    sampler=StatevectorSampler(
        default_shots=1024, seed=42,
    ),
    ansatz=small_ansatz,
    optimizer=COBYLA(maxiter=20),
    callback=record_small_energy,
)
small_result =
small_vqe.compute_minimum_eigenvalue(cost_
operator)
```

VQE STEP 5.1

INITIALIZE PARAMETERS

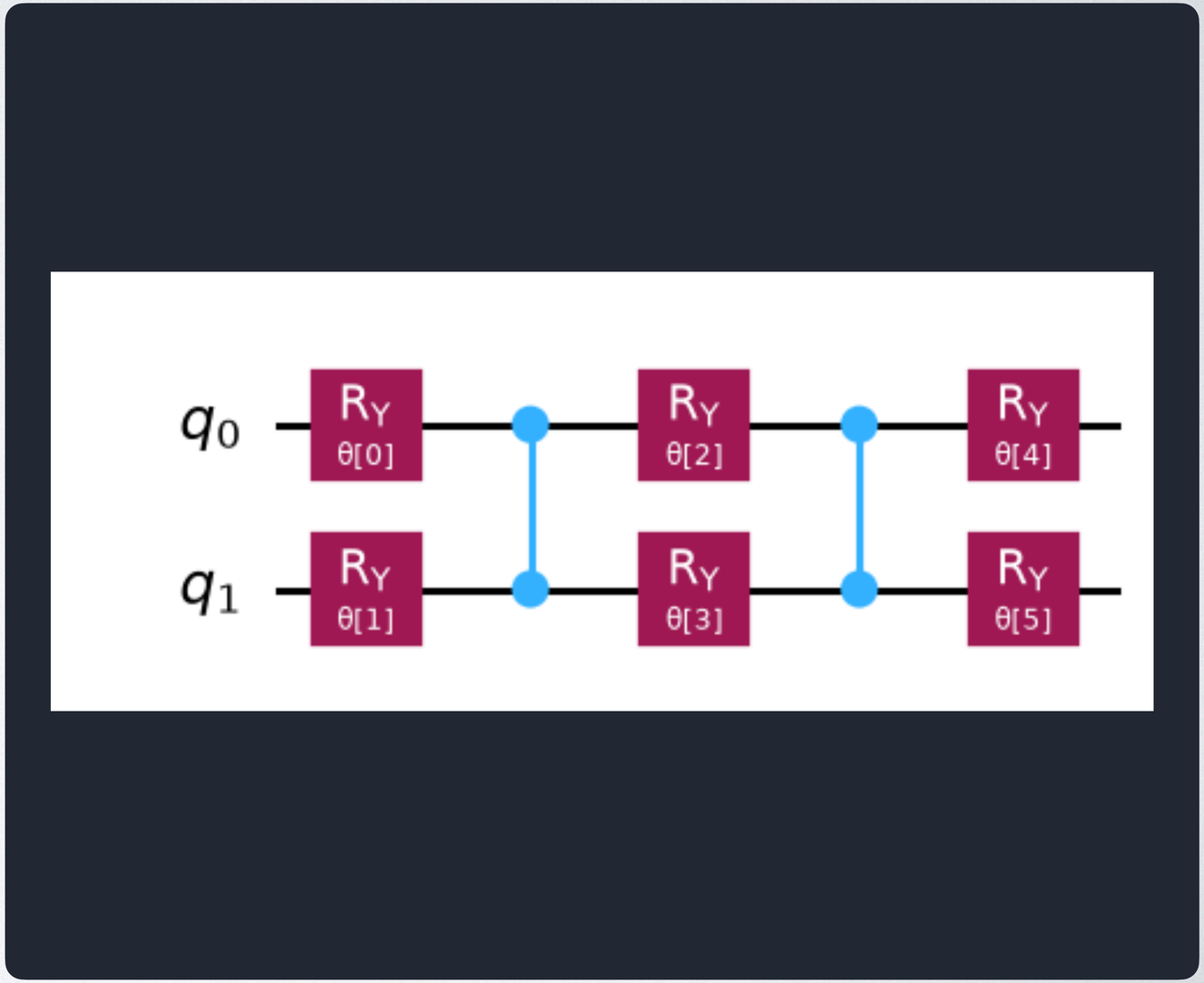
Start with an initial rotation angles θ.



VQE STEP 5.2

PREPARE QUANTUM STATE

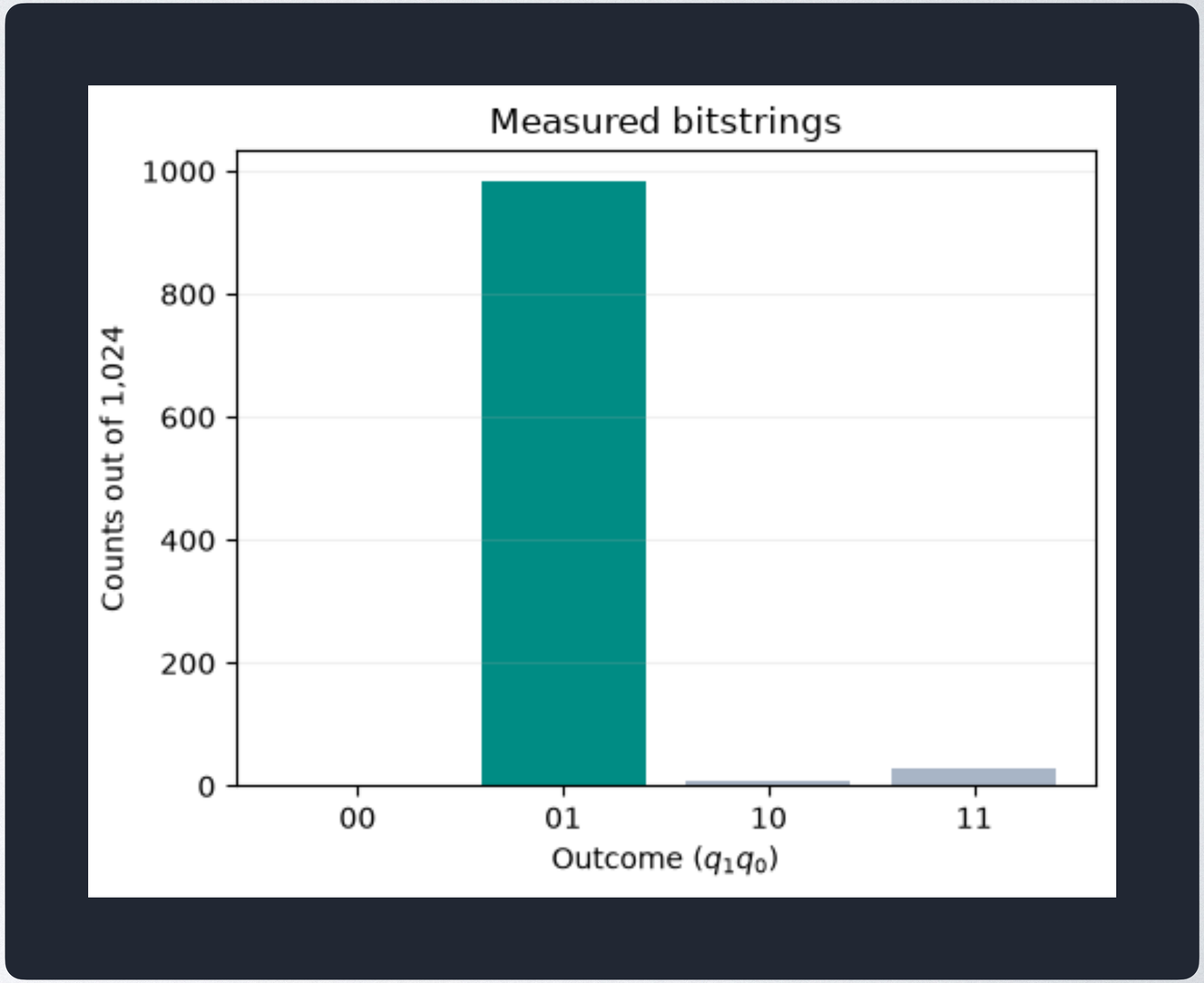
Apply the parameterized gates to prepare a candidate quantum state.



VQE STEP 5.3

MEASURE

Apply the parameterized gates to prepare a candidate quantum state.



VQE STEP 5.4

ESTIMATE ISING ENERGY

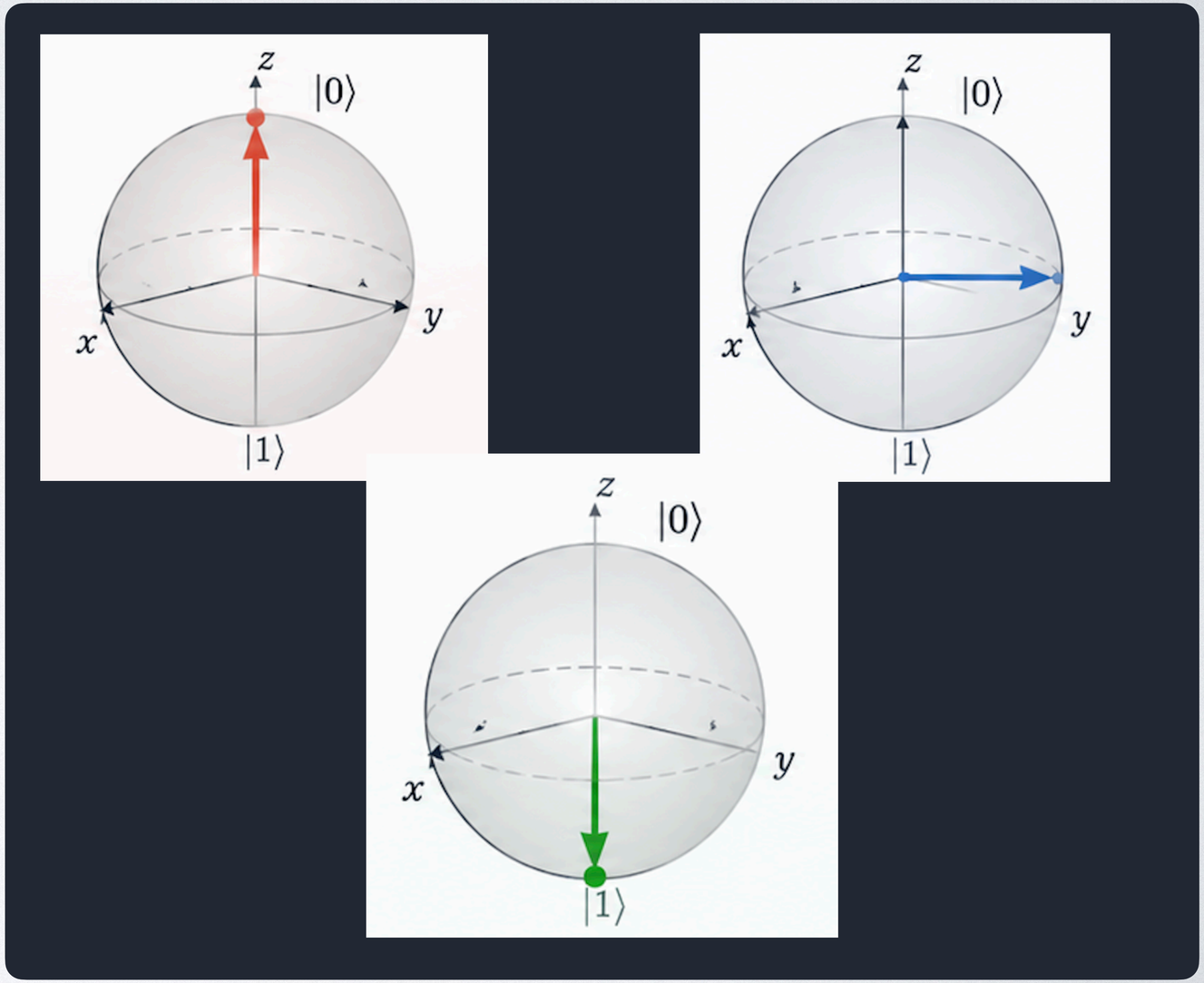
Calculate the expected energy from the measured probabilities and Hamiltonian.

$$E(\theta) = \sum_{x} P(x \mid \theta) E_H(x)$$

VQE STEP 5.5

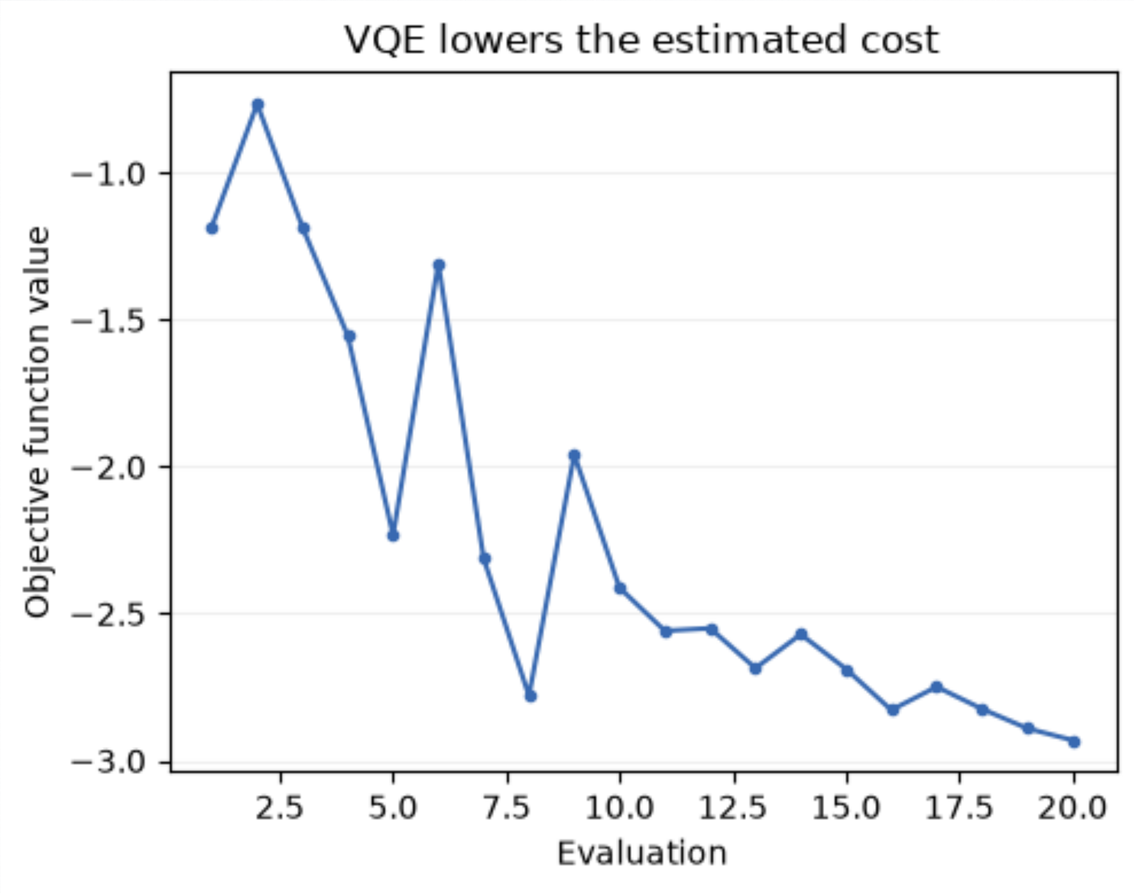
UPDATE PARAMETERS

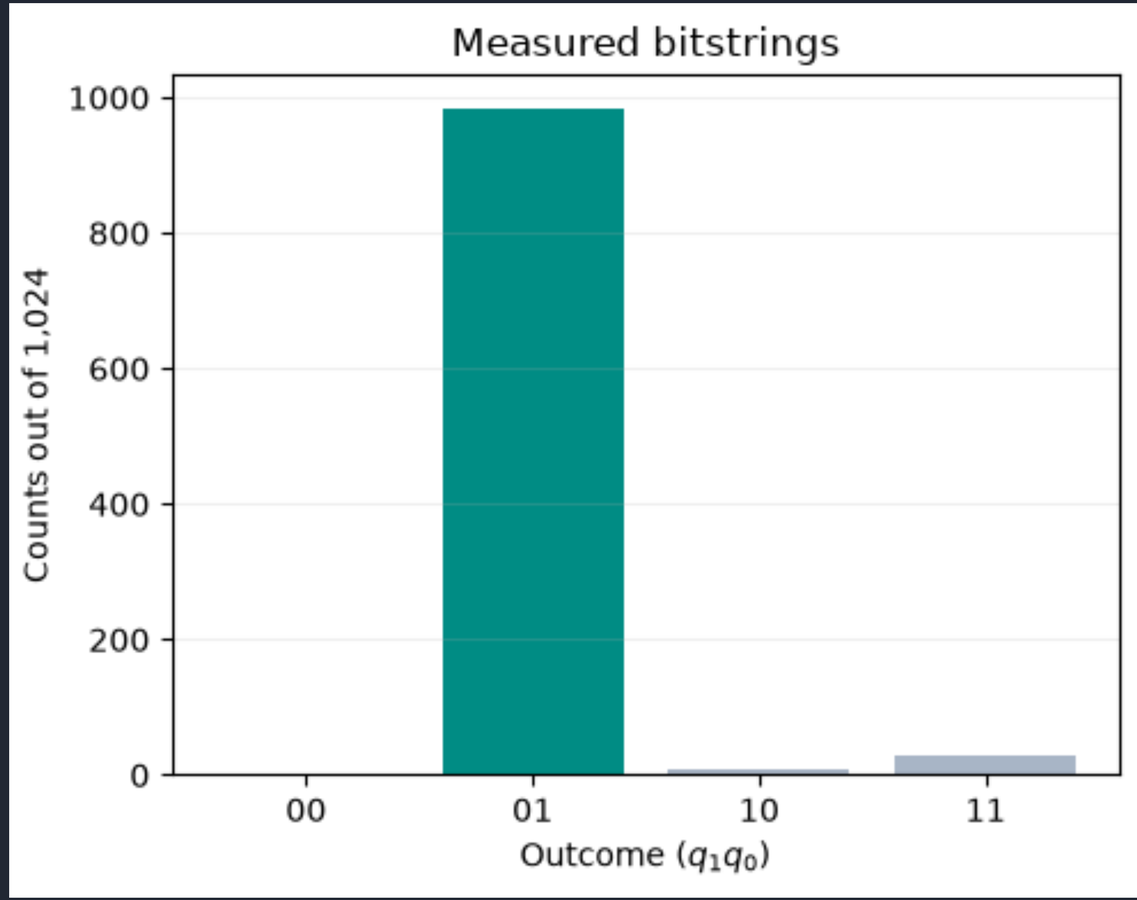
COBYLA calculate new rotation angles.



VQE STEP 6

Measure & Decode





VQE STEP 7

Measure & Decode

$$\max f(a,b) = 3a + 2b - 4ab$$

$$a, b \in 0,1$$

$$3 = 3 * 1 + 2 * 0 - 4 * 1 * 0$$

$$q0 = 1 \; q1 = 0$$

$$a = 1 \; b = 0$$

VQE ON QPU

ID	Status	Instance	Mode	Created	Completed	QPU	Usage	User	Tags
db7aegeil99s73...	Completed	open-instance	Job	10/14/26, 12:48 AM	10/14/26, 12:48 AM	ibm_fez	2s	Beilak Aliev	
db7aeemr6oqs7...	Completed	open-instance	Job	10/14/26, 12:48 AM	10/14/26, 12:48 AM	ibm_fez	2s	Beilak Aliev	
db7aecuil99s73...	Completed	open-instance	Job	10/14/26, 12:48 AM	10/14/26, 12:48 AM	ibm_fez	2s	Beilak Aliev	
db7advur6oqs7...	Completed	open-instance	Job	10/14/26, 12:47 AM	10/14/26, 12:48 AM	ibm_fez	2s	Beilak Aliev	
db7adpur6oqs7...	Completed	open-instance	Job	10/14/26, 12:47 AM	10/14/26, 12:47 AM	ibm_fez	2s	Beilak Aliev	
db7ado6r6oqs7...	Completed	open-instance	Job	10/14/26, 12:47 AM	10/14/26, 12:47 AM	ibm_fez	2s	Beilak Aliev	
db7adlqraffc73f...	Completed	open-instance	Job	10/14/26, 12:47 AM	10/14/26, 12:47 AM	ibm_fez	2s	Beilak Aliev	
db7ademil99s7...	Completed	open-instance	Job	10/14/26, 12:46 AM	10/14/26, 12:47 AM	ibm_fez	2s	Beilak Aliev	
db7adcuil99s73...	Completed	open-instance	Job	10/14/26, 12:46 AM	10/14/26, 12:46 AM	ibm_fez	2s	Beilak Aliev	
db7adb2raffc73...	Completed	open-instance	Job	10/14/26, 12:46 AM	10/14/26, 12:46 AM	ibm_fez	2s	Beilak Aliev	

Items per page: 10 | 53 items | 6 pages

db7aegeil99s73fnmkqg

Actions

Details

Edit tags

User
Beilak Aliev

Region
Washington DC (us-east)

Program
sampler

Instance
open-instance

Mode
Job

of PUBs
1

Quantum computer
ibm_fez
View calibration history

Tags

Status and usage

Status
Completed

Total completion time
7s

Total usage
2s

- Created: Oct 14, 2026, 12:48 AM
- Pending: 1s
- In progress: Oct 14, 2026, 12:48 AM
  Usage: 2s
- Completed: Oct 14, 2026, 12:48 AM

Input options

These input options apply to all PUBs within this workload.

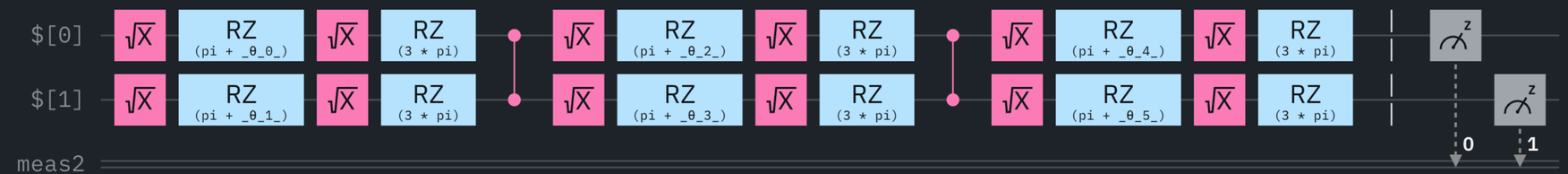

```
{ default_shots: 1024 }
default_shots: 1024
```

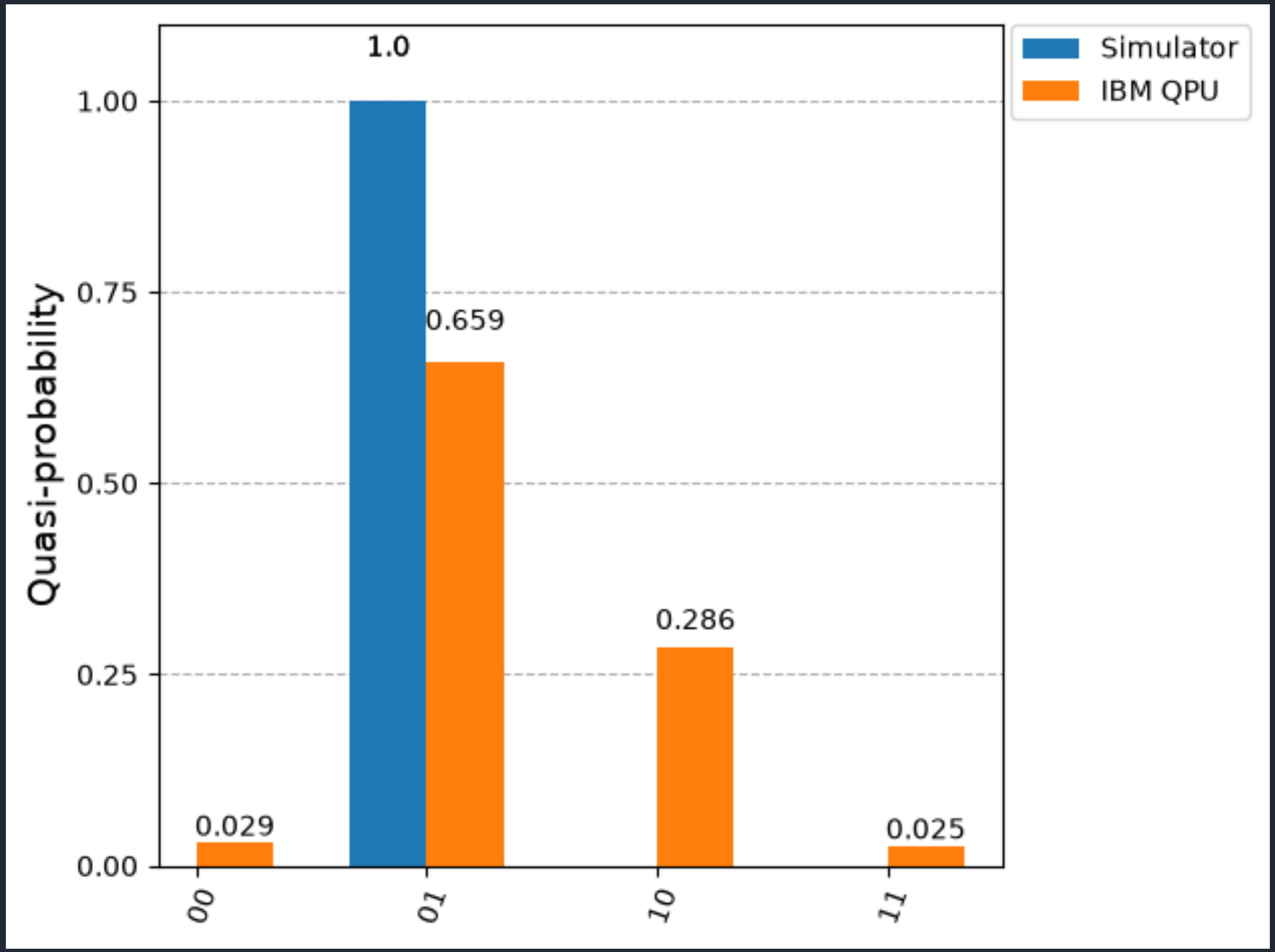
Parameter values

```
(6) [ 3.4759260722981935, -1.3244223259479924, 4.664383946278004, … ]
 0: 3.4759260722981935
 1: -1.3244223259479924
 2: 4.664383946278004
 3: 3.7736062694458186
 4: -4.903352187732255
 5: 5.54556931598153
 length: 6
```

Circuit

Diagram | Qasm | Qiskit



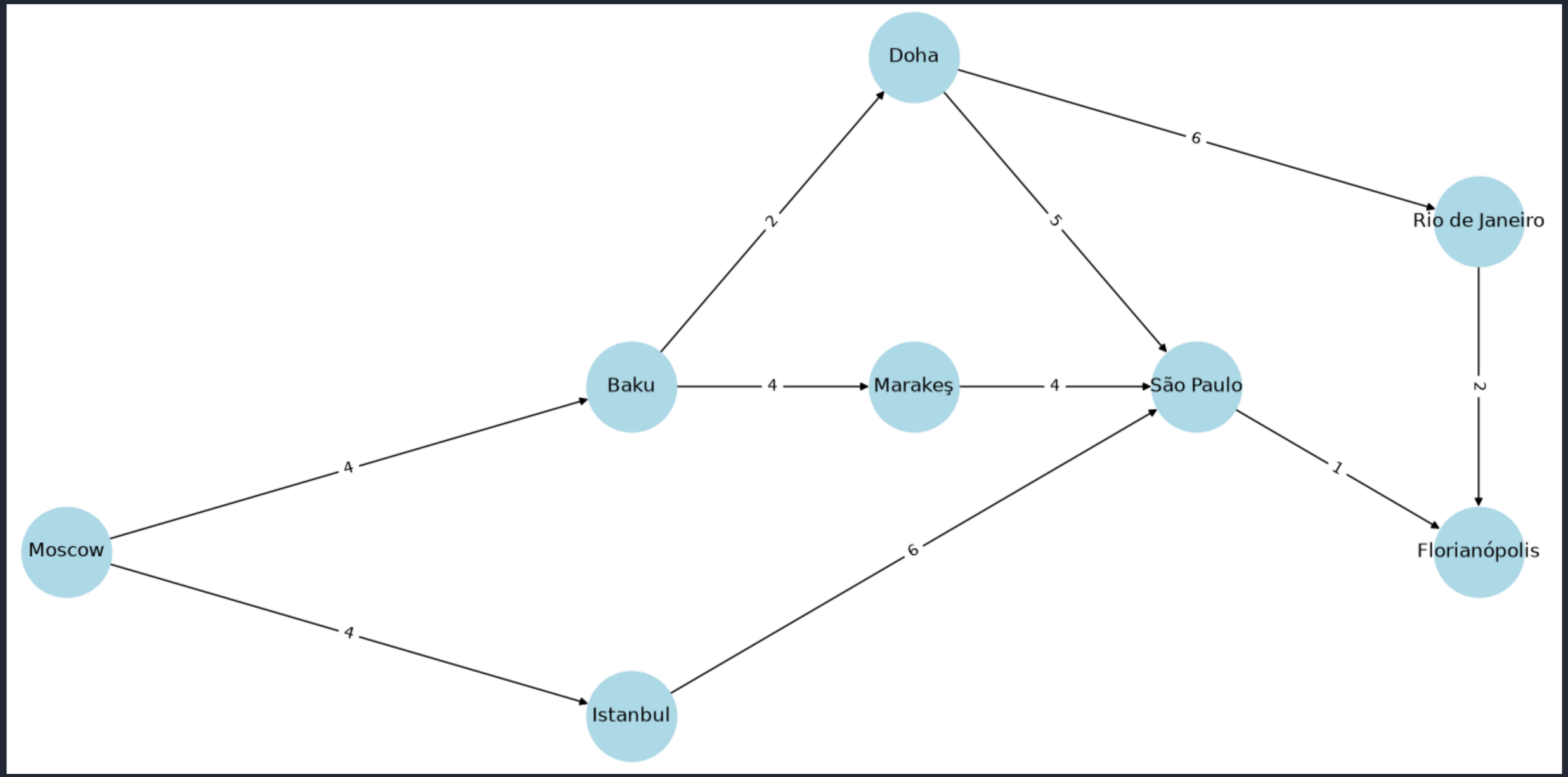

1.0
1.00
0.75
0.50
0.25
0.00
Quasi-probability
0.659
0.286
0.029
0.025
00
01
10
11
Simulator
IBM QPU

SHORTEST-PATH PROBLEM

OR

How to get from Moscow to Florianópolis?

0. PROBLEM DEFINITION



SEARCH SPACE

- 10 flights → 10 binary variables
- $2^{10} = 1{,}024$ possible combinations
- Only 4 valid routes - [We don't know this! :-)]

I. MATH FORMULATION

```
Problem name: cheapest_city_trip

Minimize
  4*x0 + 4*x1 + 2*x2 + 4*x3 + 6*x4 + 5*x5 + 6*x6 + x7 + 2*x8 + 4*x9

Subject to
  Linear constraints (8)
    x0 + x1 == 1  'flow_Moscow'
    -x0 + x2 + x3 == 0  'flow_Baku'
    -x1 + x4 == 0  'flow_Istanbul'
    -x2 + x5 + x6 == 0  'flow_Doha'
    -x4 - x5 + x7 - x9 == 0  'flow_São Paulo'
    -x6 + x8 == 0  'flow_Rio de Janeiro'
    -x7 - x8 == -1  'flow_Florianópolis'
    -x3 + x9 == 0  'flow_Marakeş'

  Binary variables (10)
    x0 x1 x2 x3 x4 x5 x6 x7 x8 x9
```

Bit	Flight	Cost
x0	Moscow → Baku	4
x1	Moscow → Istanbul	2
x2	Baku → Doha	2
x3	Baku → Marakeş	4
x4	Istanbul → São Paulo	10
x5	Doha → São Paulo	5
x6	Doha → Rio	6
x7	São Paulo → Florianópolis	1
x8	Rio → Florianópolis	2
x9	Marakeş → São Paulo	4

2.TO QUBO

```
Constraint penalty: 7
Edge variables: {'x0': 'Moscow → Baku', 'x1': 'Moscow → Istanbul', 'x2': 'Baku → Doha',
QUBO: 10 binary variables, 0 remaining constraints
Problem name: cheapest_city_trip

Minimize
  14*x0^2 + 14*x0*x1 - 14*x0*x2 - 14*x0*x3 + 14*x1^2 - 14*x1*x4 + 14*x2^2
  + 14*x2*x3 - 14*x2*x5 - 14*x2*x6 + 14*x3^2 - 14*x3*x9 + 14*x4^2 + 14*x4*x5
  - 14*x4*x7 + 14*x4*x9 + 14*x5^2 + 14*x5*x6 - 14*x5*x7 + 14*x5*x9 + 14*x6^2
  - 14*x6*x8 + 14*x7^2 + 14*x7*x8 - 14*x7*x9 + 14*x8^2 + 14*x9^2 - 10*x0 - 12*x1
  + 2*x2 + 4*x3 + 10*x4 + 5*x5 + 6*x6 - 13*x7 - 12*x8 + 4*x9 + 14

Subject to
  No constraints

  Binary variables (10)
    x0 x1 x2 x3 x4 x5 x6 x7 x8 x9
```

```python
from qiskit_optimization.converters import QuadraticProgramToQubo
converter = QuadraticProgramToQubo(penalty=7)
qubo = converter.convert(problem)
```

3. QAOA INIT

```
from qiskit.primitives import StatevectorSampler
from qiskit_algorithms import QAOA
from qiskit_algorithms.optimizers import COBYLA
from qiskit_optimization.algorithms import MinimumEigenOptimizer

qaoa = QAOA(
    sampler=StatevectorSampler(default_shots=1024, seed=42),
    optimizer=COBYLA(maxiter=100),
    reps=2)

optimyzer = MinimumEigenOptimizer(qaoa, converters=converter)
```

4. HYBRID OPTIMISATION

1. Convert QUBO to Ising
2. Execute quantum circuits
3. Measure
4. Optimise with COBYLA
5. Select the best solution

```
result = optimyzer.solve(problem)
```

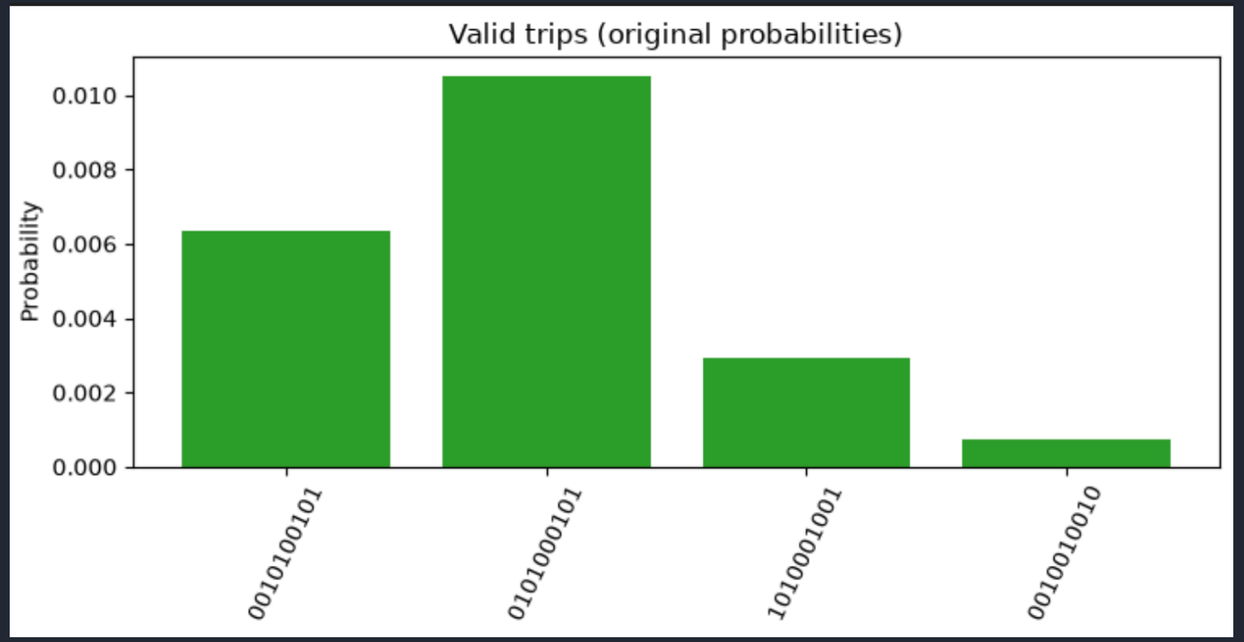
4. HYBRID OPTIMISATION

0101000101. Moscow → Baku → Doha → Rio de Janeiro → Florianópolis; cost: 14

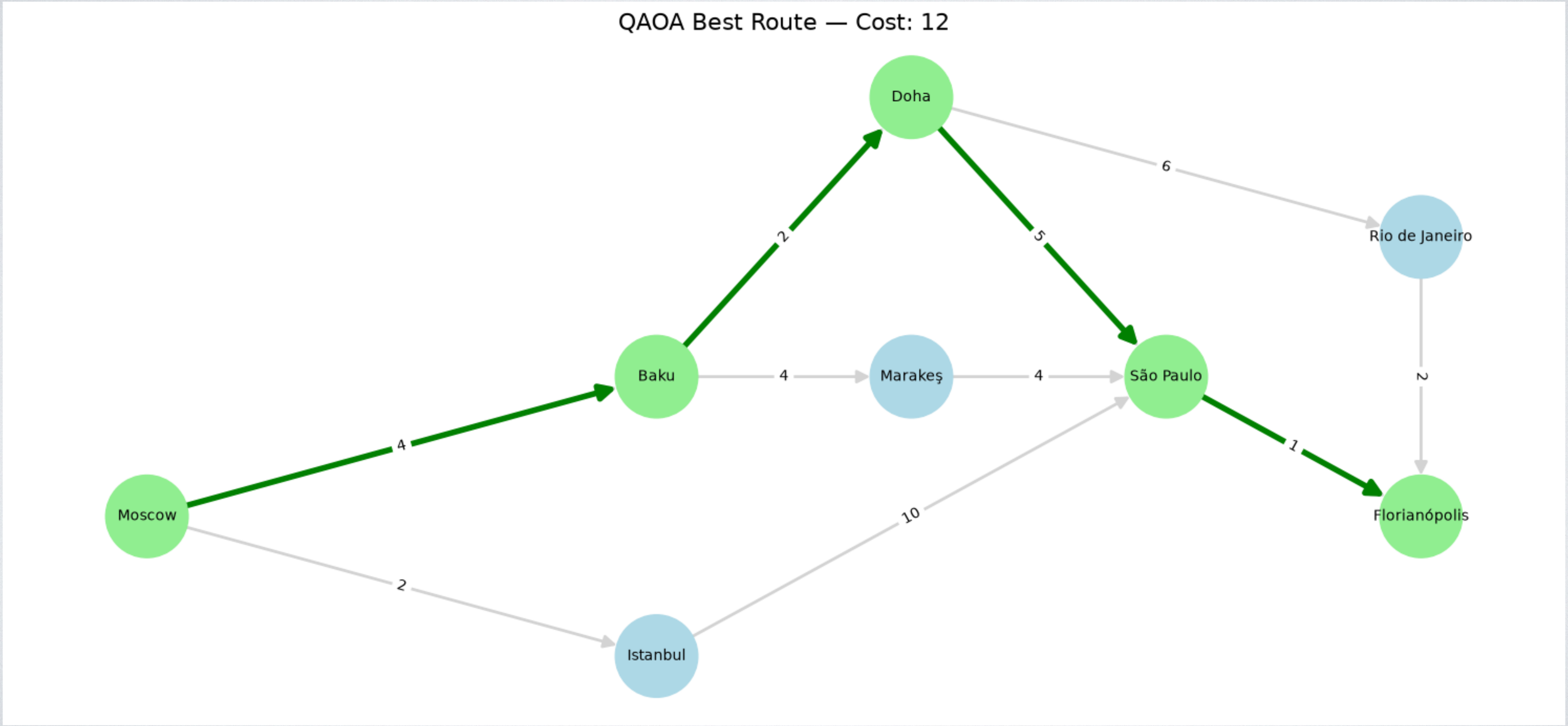
0010100101. Moscow → Baku → Doha → São Paulo → Florianópolis; cost: 12

1010001001. Moscow → Baku → Marakeş → São Paulo → Florianópolis; cost: 13

0010010010. Moscow → Istanbul → São Paulo → Florianópolis; cost: 13



How to get from Moscow to Florianópolis?



QUANTUM LEAP

BUILDING REAL QUANTUM PROGRAMS WITH PYTHON





